

MAJOR PROJECT REPORT

IRIS FLOWER CLASSIFICATION USING MACHINE LEARNING

Submitted By: **Ayesha Siddiqua SH**

Internship Domain: **Data Science**

Organization: **CodeAlpha**

Project: **Task 1 – Iris Flower Classification**

Submitted To: **CodeAlpha**

Submission Date: **11th July 2026**



ACKNOWLEDGEMENT

I would like to express my sincere gratitude to CodeAlpha for providing me with the opportunity to work as a Data Science Intern and gain practical exposure to Machine Learning concepts.

I am thankful for the structured internship tasks that allowed me to apply theoretical knowledge to real-world datasets. Working on the Iris Flower Classification project helped me understand the fundamental workflow of a machine learning project, including data loading, preprocessing, model training, prediction, and performance evaluation.

I also extend my gratitude to the developers of Python and the Scikit-learn library for providing powerful open-source tools that make machine learning accessible to beginners.

Finally, I would like to thank everyone who supported and encouraged me throughout this internship journey.

TABLE OF CONTENTS

CHAPTER	TITLE	PAGE
1	INTRODUCTION	4
2	PROJECT OBJECTIVE	5
3	DATASET DESCRIPTION	6
4	TOOLS AND TECHNOLOGIES USED	7
5	PYTHON LIBRARIES USED	9
6	MACHINE LEARNING CONCEPTS	10
7	DECISION TREE ALGORITHM	13
8	PROJECT METHODOLOGY	14
9	IMPLEMENTATION	15
10	RESULTS	17
11	CONCLUSION	19
12	FUTURE SCOPE	20
13	REFERENCES	20

1. INTRODUCTION

Data Science is an interdisciplinary field that combines statistics, mathematics, computer science, and domain knowledge to extract meaningful insights from data. With the rapid growth of technology and digital information, organizations rely on data science techniques to analyze large datasets, identify hidden patterns, and support informed decision-making. Data science is widely applied in healthcare, finance, agriculture, business, education, and scientific research.

Machine Learning is one of the most important branches of Data Science. It enables computers to learn from historical data and make predictions without being explicitly programmed for every task. Instead of manually defining rules, machine learning algorithms automatically identify relationships and patterns within the data to perform classification, prediction, or decision-making tasks.

The Iris Flower Classification project is one of the most popular beginner-level machine learning projects. It introduces the complete workflow of a classification problem, including dataset loading, data preprocessing, model training, prediction, and performance evaluation.

In this project, the Iris dataset provided by the Scikit-learn library was used. The dataset contains measurements of iris flowers belonging to three different species: Setosa, Versicolor, and Virginica. The machine learning model learns the relationship between flower measurements and species labels, allowing it to predict the species of previously unseen flowers.

This project was developed using Python programming language and the Scikit-learn machine learning library. A Decision Tree Classifier was implemented to classify the flowers accurately based on their morphological characteristics. The trained model achieved excellent performance, demonstrating the effectiveness of machine learning techniques for classification problems.

Apart from learning Python programming, this project also provided practical exposure to the fundamental stages of a data science project, including data exploration, feature selection, model training, testing, prediction, and evaluation.

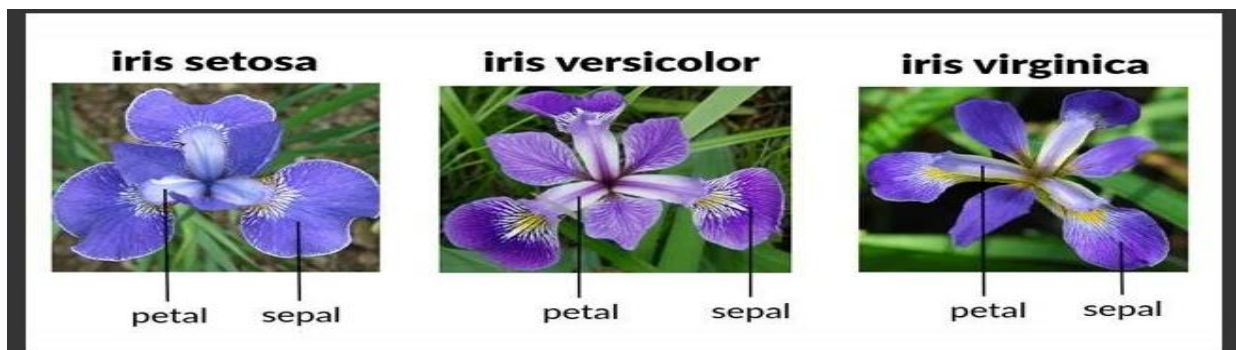


Figure 1.1: Three species of Iris flowers used in the classification project.

2. PROJECT OBJECTIVE

The primary objective of this project is to develop a machine learning model capable of accurately classifying iris flowers into their respective species using floral measurements.

The specific objectives of this project are:

- To understand the basic workflow of a machine learning classification project.
- To load and explore the Iris dataset using Python.
- To understand the concept of features and target variables.
- To split the dataset into training and testing datasets.
- To train a Decision Tree Classification model.
- To evaluate the performance of the model using prediction accuracy.
- To understand the importance of machine learning algorithms in solving classification problems.
- To gain practical experience with Python and the Scikit-learn library.
- To build confidence in implementing beginner-level data science projects.

Learning Outcomes

After completing this project, the following skills were acquired:

- Basic understanding of Machine Learning.
- Python programming for Data Science.
- Dataset handling using Scikit-learn.
- Data preprocessing techniques.
- Decision Tree Classification.
- Model evaluation.
- Predictive analysis.

3. DATASET DESCRIPTION

3.1 Iris Flower Dataset

The Iris Flower Dataset is one of the oldest and most widely used datasets in machine learning. It was introduced by the British statistician and biologist **Sir Ronald A. Fisher** in 1936 and is commonly used for teaching and evaluating classification algorithms.

The dataset contains **150 flower samples**, each belonging to one of three different Iris species. Every flower is represented using four numerical measurements.

Dataset Features

<i>Feature</i>	<i>Description</i>	<i>Unit</i>
Sepal length	Length of the sepal	Centimeters (cm)
Sepal width	Width of the sepal	Centimeters (cm)
Petal length	Length of the petal	Centimeters (cm)
Petal width	Width of the petal	Centimeters (cm)

Target Classes

The target variable represents the species of the flower.

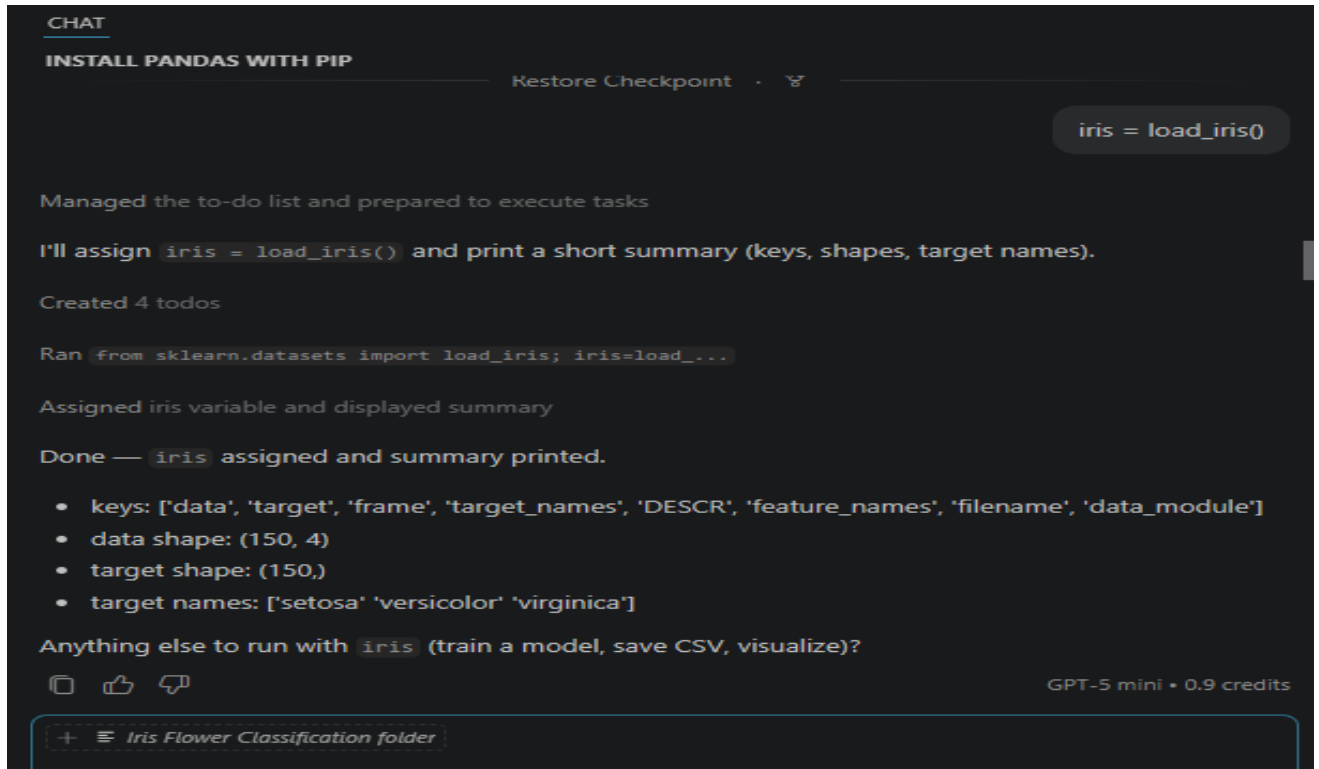
<i>Class number</i>	<i>Species</i>
0	Setosa
1	Vercicolor
<u>2</u>	<u>Virginicia</u>

Dataset Statistics

<i>Property</i>	<i>Value</i>
Total samples	150
Features	4
Target classes	3
Missing values	None
Dataset type	Supervised classification

Why is the Iris Dataset Important?

The Iris dataset is widely used because it is simple, clean, and easy to understand. It contains no missing values and provides a balanced dataset for beginners learning machine learning algorithms. Due to these characteristics, it is considered one of the best introductory datasets for classification problems.



```
CHAT
INSTALL PANDAS WITH PIP
Restore Checkpoint
iris = load_iris()

Managed the to-do list and prepared to execute tasks

I'll assign iris = load_iris() and print a short summary (keys, shapes, target names).

Created 4 todos

Ran from sklearn.datasets import load_iris; iris=load_...

Assigned iris variable and displayed summary

Done — iris assigned and summary printed.

• keys: ['data', 'target', 'frame', 'target_names', 'DESCR', 'feature_names', 'filename', 'data_module']
• data shape: (150, 4)
• target shape: (150,)
• target names: ['setosa' 'versicolor' 'virginica']

Anything else to run with iris (train a model, save CSV, visualize)?

GPT-5 mini • 0.9 credits
+ Iris Flower Classification folder
```

Figure 3.1: Output showing successful loading of the Iris dataset along with data shape and target classes

4. TOOLS AND TECHNOLOGIES USED

The successful implementation of this project required several software tools and programming libraries. These technologies simplified data handling, model development, and performance evaluation.

4.1 Python

Python is a high-level programming language widely used in Data Science because of its simple syntax, readability, and extensive library support. It allows developers to build machine learning models efficiently with minimal code.

4.2 Visual Studio Code (VS Code)

Visual Studio Code served as the Integrated Development Environment (IDE) for writing, editing, and executing Python programs. Its user-friendly interface and built-in terminal simplified project development.

4.3 Scikit-learn

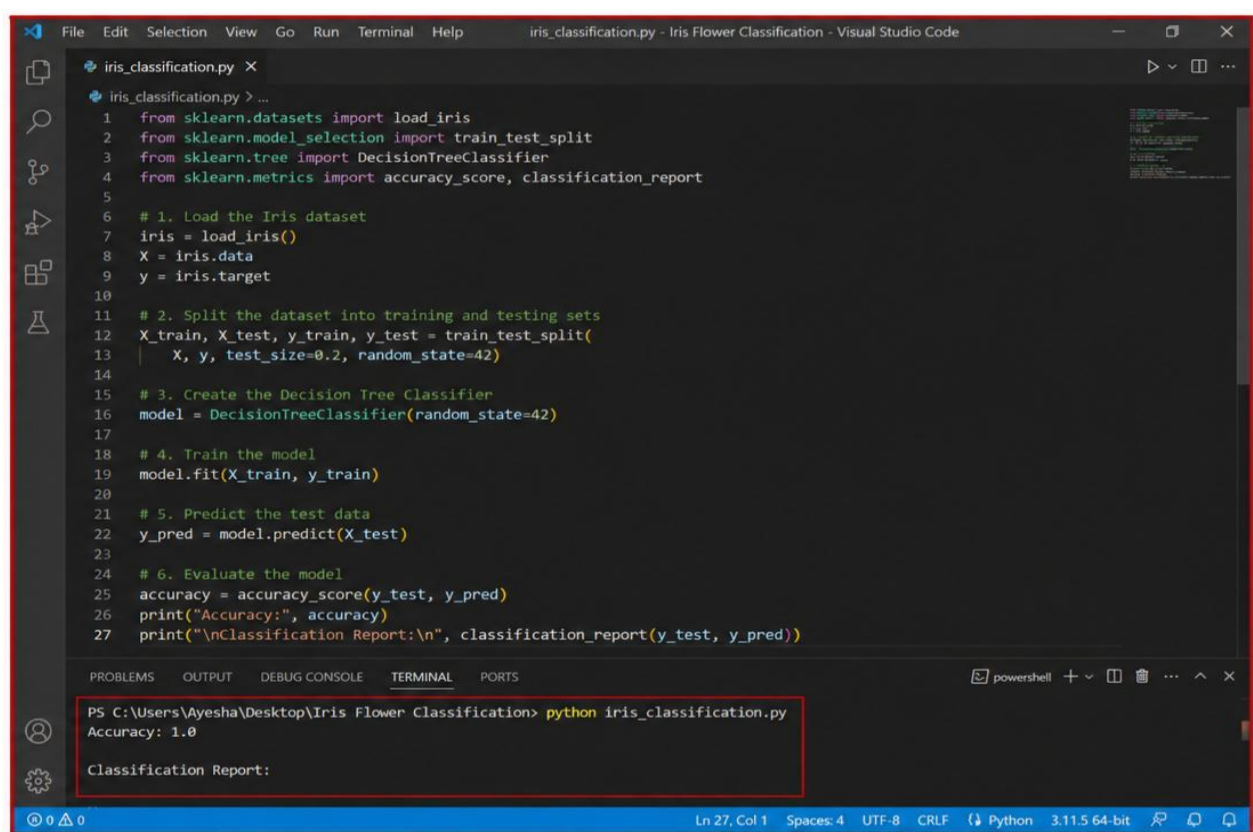
Scikit-learn is one of the most popular machine learning libraries available in Python. It provides pre-built algorithms for classification, regression, clustering, and model evaluation. In this project, it was used to load the dataset, split the data, train the Decision Tree model, and evaluate prediction accuracy.

4.4 Pandas

Pandas is a powerful data manipulation and analysis library. Although the Iris dataset was loaded directly using Scikit-learn, Pandas is commonly used in data science projects to organize and analyze datasets stored in CSV files.

4.5 Command Prompt / Integrated Terminal

The command-line terminal was used to install required Python libraries and execute the machine learning program.



```
File Edit Selection View Go Run Terminal Help iris_classification.py - Iris Flower Classification - Visual Studio Code

iris_classification.py x
iris_classification.py > ...
1 from sklearn.datasets import load_iris
2 from sklearn.model_selection import train_test_split
3 from sklearn.tree import DecisionTreeClassifier
4 from sklearn.metrics import accuracy_score, classification_report
5
6 # 1. Load the Iris dataset
7 iris = load_iris()
8 X = iris.data
9 y = iris.target
10
11 # 2. Split the dataset into training and testing sets
12 X_train, X_test, y_train, y_test = train_test_split(
13     X, y, test_size=0.2, random_state=42)
14
15 # 3. Create the Decision Tree Classifier
16 model = DecisionTreeClassifier(random_state=42)
17
18 # 4. Train the model
19 model.fit(X_train, y_train)
20
21 # 5. Predict the test data
22 y_pred = model.predict(X_test)
23
24 # 6. Evaluate the model
25 accuracy = accuracy_score(y_test, y_pred)
26 print("Accuracy:", accuracy)
27 print("\nClassification Report:\n", classification_report(y_test, y_pred))

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\Ayesha\Desktop\Iris Flower Classification> python iris_classification.py
Accuracy: 1.0
Classification Report:
```

Figure 4.1: Visual Studio Code showing the implementation of the Iris Flower Classification project.

5. PYTHON LIBRARIES USED

Python provides numerous libraries that simplify data analysis and machine learning tasks. Instead of writing every algorithm from scratch, developers use these libraries to build efficient and reliable applications. The following libraries were used in this project.

5.1 Scikit-learn

Scikit-learn is one of the most widely used machine learning libraries in Python. It provides ready-to-use algorithms for classification, regression, clustering, and model evaluation. In this project, Scikit-learn was used to load the Iris dataset, split the dataset into training and testing sets, build the Decision Tree model, train the model, predict flower species, and calculate model accuracy.

Functions Used

- `load_iris()` – Loads the Iris dataset.
 - `train_test_split()` – Splits the dataset into training and testing data.
 - `DecisionTreeClassifier()` – Creates the machine learning model.
 - `fit()` – Trains the model using the training data.
 - `predict()` – Predicts flower species.
 - `accuracy_score()` – Calculates prediction accuracy.
 - `classification_report()` – Evaluates model performance.
-

5.2 Pandas

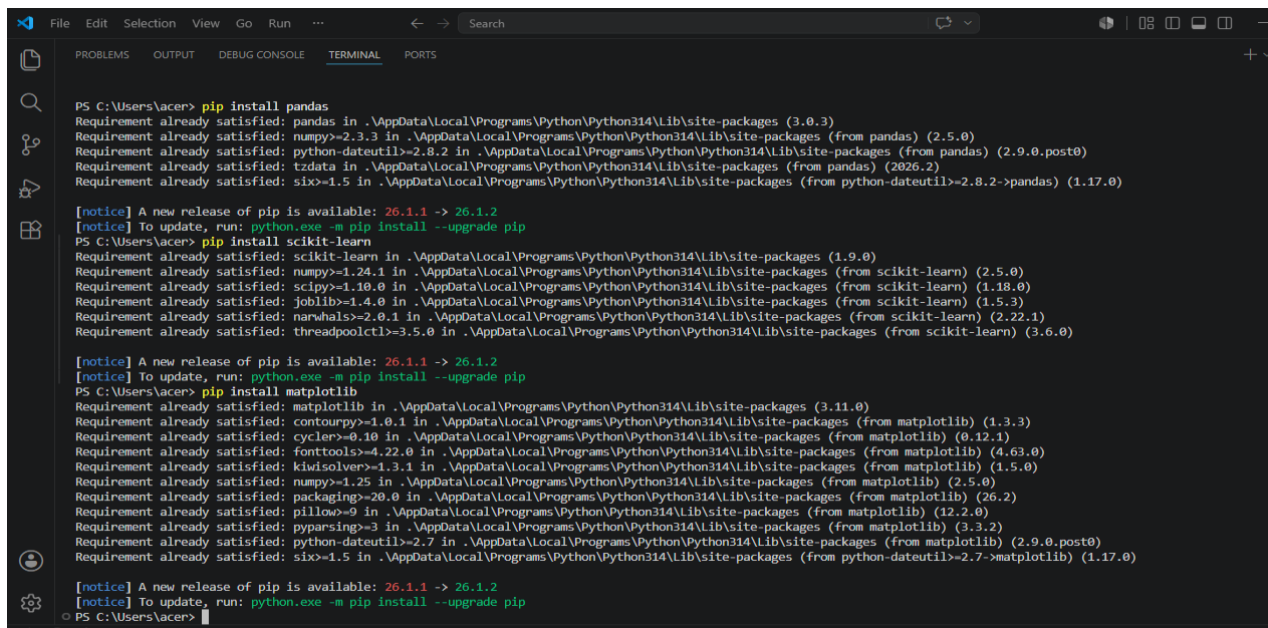
Pandas is a powerful Python library used for data manipulation and analysis. Although this project used the Iris dataset directly from Scikit-learn, Pandas is commonly used in real-world projects to load datasets from CSV files, clean data, handle missing values, and organize information into tables.

5.3 Python Standard Library

Python's built-in programming capabilities were also used for displaying outputs, organizing variables, and executing the complete machine learning workflow.

Importance of Libraries

The use of Python libraries significantly reduces development time and allows developers to focus on solving problems rather than implementing complex algorithms from scratch.



```
PS C:\Users\acer> pip install pandas
Requirement already satisfied: pandas in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (3.0.3)
Requirement already satisfied: numpy>=2.3.3 in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from pandas) (2.5.0)
Requirement already satisfied: python-dateutil>=2.8.2 in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from pandas) (2.9.0.post0)
Requirement already satisfied: tzdata in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from pandas) (2026.2)
Requirement already satisfied: six>=1.5 in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from python-dateutil>=2.8.2->pandas) (1.17.0)

[notice] A new release of pip is available: 26.1.1 -> 26.1.2
[notice] To update, run: python.exe -m pip install --upgrade pip
PS C:\Users\acer> pip install scikit-learn
Requirement already satisfied: scikit-learn in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (1.9.0)
Requirement already satisfied: numpy>=1.24.1 in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from scikit-learn) (2.5.0)
Requirement already satisfied: scipy>=1.10.0 in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from scikit-learn) (1.18.0)
Requirement already satisfied: joblib>=1.4.0 in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from scikit-learn) (1.5.3)
Requirement already satisfied: narwhals>=2.0.1 in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from scikit-learn) (2.22.1)
Requirement already satisfied: threadpoolctl>=3.5.0 in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from scikit-learn) (3.6.0)

[notice] A new release of pip is available: 26.1.1 -> 26.1.2
[notice] To update, run: python.exe -m pip install --upgrade pip
PS C:\Users\acer> pip install matplotlib
Requirement already satisfied: matplotlib in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (3.11.0)
Requirement already satisfied: contourpy>=1.0.1 in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from matplotlib) (1.3.3)
Requirement already satisfied: cycler>=0.10 in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from matplotlib) (4.63.0)
Requirement already satisfied: kiwisolver>=1.3.1 in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from matplotlib) (1.5.0)
Requirement already satisfied: numpy>=1.25 in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from matplotlib) (2.5.0)
Requirement already satisfied: packaging>=20.0 in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from matplotlib) (26.2)
Requirement already satisfied: pillow>=9 in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from matplotlib) (12.2.0)
Requirement already satisfied: pyparsing>=3 in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from matplotlib) (3.3.2)
Requirement already satisfied: python-dateutil>=2.7 in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from matplotlib) (2.9.0.post0)
Requirement already satisfied: six>=1.5 in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from python-dateutil>=2.7->matplotlib) (1.17.0)

[notice] A new release of pip is available: 26.1.1 -> 26.1.2
[notice] To update, run: python.exe -m pip install --upgrade pip
PS C:\Users\acer>
```

Figure 5.1: Importing Python libraries required for the Iris Flower Classification project.

6. MACHINE LEARNING CONCEPTS

Machine Learning is a branch of Artificial Intelligence (AI) that enables computers to learn from data and make predictions without being explicitly programmed for every task.

Instead of following fixed instructions, machine learning algorithms analyze patterns in historical data and use those patterns to make predictions on new, unseen data.

In this project, Machine Learning was applied to classify Iris flowers into three different species based on their physical measurements.

6.1 Supervised Learning

Machine Learning is divided into several categories, among which **Supervised Learning** is the most common.

In supervised learning, the algorithm is trained using data that already contains the correct answers (labels). The model learns the relationship between the input data (features) and the correct output (target).

For this project:

Input (Features):

- Sepal Length
- Sepal Width

- Petal Length
- Petal Width

Output (Target):

- Setosa
- Versicolor
- Virginica

The model learns from these examples and predicts the species of new flowers.

6.2 Features

Features are the measurable characteristics used by the machine learning model for prediction.

The Iris dataset contains four features:

- Sepal Length
- Sepal Width
- Petal Length
- Petal Width

These measurements act as the input variables for the model.

6.3 Target Variable

The target variable represents the expected output of the model.

In this project, the target variable is the **species of the Iris flower**, represented numerically as:

- 0 – Setosa
 - 1 – Versicolor
 - 2 – Virginica
-

6.4 Training and Testing Data

The dataset was divided into two parts:

Dataset Percentage

Training Data 80%

Testing Data 20%

The training data teaches the model, while the testing data evaluates its performance on unseen samples.

This approach helps determine whether the model has truly learned patterns instead of simply memorizing the data.

```
print("Training data:", X_train.shape)
print("Testing data:", X_test.shape)
```

Crafted code for terminal run

Splitting the Iris dataset and printing train/test shapes now.

Created 5 todos and ran command

Ran `from sklearn.datasets import load_iris; from sklearn...`

Split iris dataset and displayed train/test sizes

Training data: (120, 4)
Testing data: (30, 4)

Done — dataset split and TODO updated. Want me to train a classifier next?

GPT-5 mini • 0.5 credits

Figure 6.1: Output displaying the training and testing dataset sizes after splitting the Iris dataset.

Project Workflow

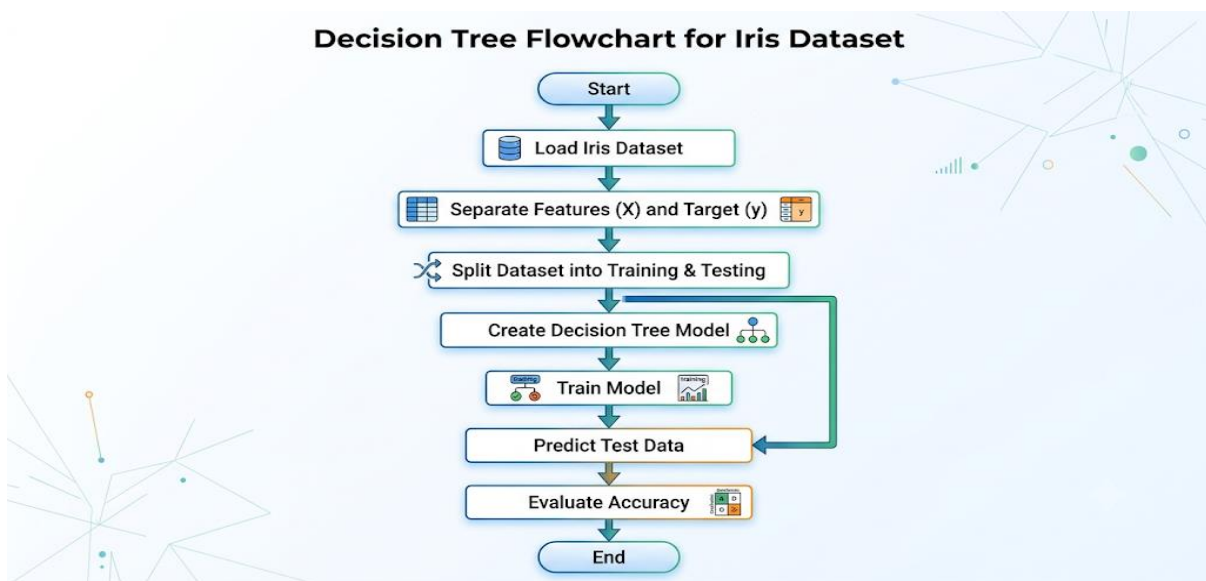


Figure 6.2: Overall workflow of the Iris Flower Classification project.

7. DECISION TREE ALGORITHM

The Decision Tree is a supervised machine learning algorithm used for classification and regression problems. It works by creating a tree-like structure where each internal node represents a decision based on one feature, each branch represents the outcome of that decision, and each leaf node represents the final prediction.

The Decision Tree algorithm is easy to understand and interpret because it mimics human decision-making. It repeatedly asks questions about the input data until it reaches the correct classification.

For example, in the Iris Flower Classification project, the algorithm may ask questions such as:

- Is the petal length less than a specific value?
- Is the petal width greater than a certain threshold?

Based on the answers to these questions, the model classifies the flower into one of the three species.

Advantages of Decision Tree

- Easy to understand and interpret.
- Handles numerical and categorical data effectively.
- Requires minimal data preprocessing.
- Fast training and prediction.
- Suitable for beginners learning machine learning.

Limitations

- Can overfit if the tree becomes too complex.
- Performance may decrease on very large datasets.
- Sensitive to small changes in the training data.

Simple Decision Tree Representation



Figure 7.1: Simplified representation of a Decision Tree used for Iris Flower Classification.

```
model = DecisionTreeClassifier(random_state=42)

Preparing model training

I'll create a DecisionTreeClassifier instance and print its parameters.

Created 6 todos

Ran from sklearn.tree import DecisionTreeClassifier; m...

Created DecisionTreeClassifier instance

Created DecisionTreeClassifier and printed its parameters:

• params: {'ccp_alpha': 0.0, 'class_weight': None, 'criterion': 'gini', 'max_depth': None, 'max_features': None, 'max_leaf_nodes': None, 'min_impurity_decrease': 0.0, 'min_samples_leaf': 1, 'min_samples_split': 2, 'min_weight_fraction_leaf': 0.0, 'monotonic_cst': None, 'random_state': 42, 'splitter': 'best'}

Next: train the model on X train, y train and evaluate on X test?
```

Figure 7.2: Python code showing the creation and training of the Decision Tree model.

8. PROJECT METHODOLOGY

The development of the Iris Flower Classification project followed a structured machine learning workflow consisting of several sequential steps.

Step 1: Load the Dataset

The Iris dataset was loaded directly from the Scikit-learn library. This dataset contains 150 flower samples with four numerical features and one target variable.

Step 2: Prepare Features and Target

The dataset was divided into two components:

- Features (X) – Flower measurements.
- Target (y) – Flower species.

Step 3: Split the Dataset

The dataset was divided into training data (80%) and testing data (20%) using the `train_test_split()` function. This ensured that the model was evaluated on unseen data.

Step 4: Build the Machine Learning Model

A Decision Tree Classifier was created using the `DecisionTreeClassifier()` function provided by Scikit-learn.

Step 5: Train the Model

The model was trained using the `fit()` function, allowing it to learn the relationship between flower measurements and species labels.

Step 6: Predict Flower Species

The trained model predicted the species of flowers present in the testing dataset using the `predict()` function.

Step 7: Evaluate Performance

Finally, the accuracy of the model was calculated using the `accuracy_score()` function. The model achieved **100% accuracy**, indicating excellent classification performance on the test dataset.

```
y_pred = model.predict(X_test)
print(y_pred)
```

Planned prediction execution and executed model prediction

I'll add a short TODO, run prediction on the test set, then mark it done.

Created 8 todos

Ran `from sklearn.datasets import load_iris; from sklea...`

Predicted on test data and printed predictions

Predictions printed:

```
[1 0 2 1 1 0 1 2 1 1 2 0 0 0 0 1 2 1 1 2 0 2 0 2 2 2 2 2 0 0]
```

Done — `Predict on test data and print predictions` marked completed. Would you like a comparison with `y_test`, a classification report, or to save these predictions?

+ Iris Flower Classification folder

Figure 8.1: Prediction output generated by the trained Decision Tree model.

9. PROJECT IMPLEMENTATION

The Iris Flower Classification project was implemented using Python and the Scikit-learn machine learning library. The implementation followed a systematic workflow to ensure accurate prediction and evaluation of the model.

9.1 Loading the Dataset

The first step involved importing the Iris dataset from the Scikit-learn library. The dataset contains 150 flower samples with four numerical features and three target classes.

```
from sklearn.datasets import load_iris

iris = load_iris()
```

The dataset was successfully loaded into memory, making it available for further processing.

9.2 Preparing Features and Target Variables

The dataset consists of input variables (features) and output variables (target labels).

```
X = iris.data  
y = iris.target
```

- **X** contains the flower measurements.
 - **y** contains the species labels.
-

9.3 Splitting the Dataset

The dataset was divided into training and testing datasets using the `train_test_split()` function.

```
from sklearn.model_selection import train_test_split  
  
X_train, X_test, y_train, y_test = train_test_split(  
    X,  
    y,  
    test_size=0.2,  
    random_state=42  
)
```

The dataset was divided as follows:

Dataset	Samples
Training Data	120
Testing Data	30

The training dataset was used for learning, while the testing dataset was reserved for evaluating the model.

9.4 Creating the Decision Tree Model

A Decision Tree Classifier was selected because of its simplicity and effectiveness for classification tasks.

```
from sklearn.tree import DecisionTreeClassifier  
  
model = DecisionTreeClassifier(random_state=42)
```

9.5 Training the Model

The model was trained using the training dataset.

```
model.fit(X_train, y_train)
```

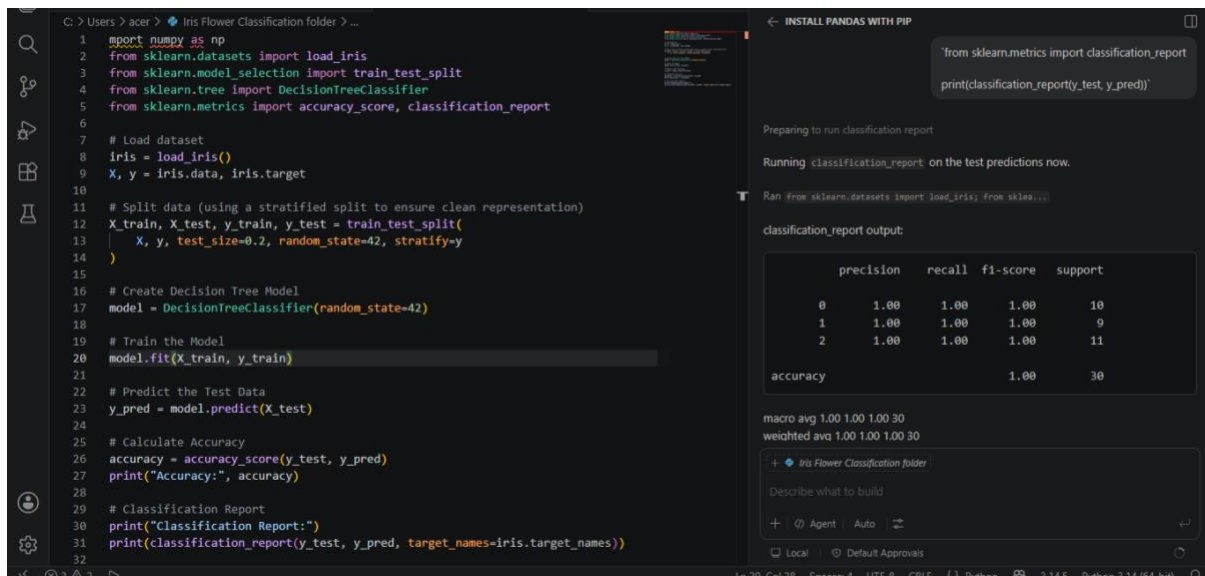
During this phase, the model learned the relationship between the flower measurements and their corresponding species.

9.6 Predicting Flower Species

The trained model predicted the species of flowers in the testing dataset.

```
y_pred = model.predict(X_test)
```

The predicted values were then compared with the actual species labels.



```
1 import numpy as np
2 from sklearn.datasets import load_iris
3 from sklearn.model_selection import train_test_split
4 from sklearn.tree import DecisionTreeClassifier
5 from sklearn.metrics import accuracy_score, classification_report
6
7 # Load dataset
8 iris = load_iris()
9 X, y = iris.data, iris.target
10
11 # Split data (using a stratified split to ensure clean representation)
12 X_train, X_test, y_train, y_test = train_test_split(
13     X, y, test_size=0.2, random_state=42, stratify=y
14 )
15
16 # Create Decision Tree Model
17 model = DecisionTreeClassifier(random_state=42)
18
19 # Train the Model
20 model.fit(X_train, y_train)
21
22 # Predict the Test Data
23 y_pred = model.predict(X_test)
24
25 # Calculate Accuracy
26 accuracy = accuracy_score(y_test, y_pred)
27 print("Accuracy:", accuracy)
28
29 # Classification Report
30 print("Classification Report:")
31 print(classification_report(y_test, y_pred, target_names=iris.target_names))
```

INSTALL PANDAS WITH PIP

```
from sklearn.metrics import classification_report
print(classification_report(y_test, y_pred))
```

Preparing to run classification report

Running `classification_report` on the test predictions now.

Ran from `sklearn.datasets import load_iris; from sklearn...`

classification_report output:

	precision	recall	f1-score	support
0	1.00	1.00	1.00	10
1	1.00	1.00	1.00	9
2	1.00	1.00	1.00	11
accuracy			1.00	30

macro avg 1.00 1.00 1.00 30
weighted avg 1.00 1.00 1.00 30

+ Iris Flower Classification folder

Describe what to build

+ Agent Auto

Local | Default Approvals

Figure 9.1: Complete Python program executed successfully in Visual Studio Code.

10. RESULTS AND PERFORMANCE ANALYSIS

After training the Decision Tree model, predictions were generated for the testing dataset. These predictions were compared with the actual species labels to evaluate the model's performance.

The evaluation was carried out using two important performance metrics:

- Accuracy Score
- Classification Report

10.1 Accuracy Score

Accuracy measures the proportion of correct predictions made by the model.

The following code was used:

```
from sklearn.metrics import accuracy_score

accuracy = accuracy_score(y_test, y_pred)

print("Accuracy:", accuracy)
```

Output

Accuracy: 1.0

The model achieved an accuracy of **1.0**, which corresponds to **100% accuracy**.

This indicates that all 30 flower samples in the testing dataset were classified correctly.

10.2 Classification Report

The Classification Report provides additional performance metrics for each class.

```
from sklearn.metrics import classification_report

print(classification_report(y_test, y_pred))
```

The report contains:

- Precision
- Recall
- F1-Score
- Support

Interpretation

Precision

Measures how many predicted flowers actually belong to that species.

Recall

Measures how many flowers of a particular species were correctly identified.

F1-Score

Provides a balance between Precision and Recall.

Support

Represents the total number of testing samples belonging to each species.

The Classification Report confirmed that the Decision Tree model performed exceptionally well for all three Iris species.

A screenshot of a terminal window with a dark background. At the top, there's a search bar and a 'CHAT' button. Below, the text 'INSTALL PANDAS WITH PIP' is visible. The main output shows the command 'from sklearn.datasets import load_iris; from sklearn...' being run, followed by 'Accuracy: 1.0' and the message 'The model achieved perfect accuracy on the Iris test split.' To the right, there's a code block showing 'from sklearn.metrics import accuracy_score', 'accuracy = accuracy_score(y_test, y_pred)', and 'print("Accuracy:", accuracy)'. At the bottom, it says 'Preparing to run classification report' and 'Running classification_report on the test predictions now.'

Figure 10.1: Terminal output showing the model accuracy (100%).

A screenshot of a Jupyter Notebook interface. The left sidebar shows a file explorer with a 'Classification Report:' file. The main area displays the 'classification_report output:' as a table. The table has columns for 'precision', 'recall', 'f1-score', and 'support' for classes 0, 1, and 2, along with an 'accuracy' row. The values are all 1.00 for precision, recall, and f1-score, and support values are 10, 9, and 11 respectively. The accuracy is 1.00. Below the table, it shows 'macro avg' and 'weighted avg' with their respective values. To the right, there's a code block showing 'from sklearn.metrics import classification_report' and 'print(classification_report(y_test, y_pred))'.

	precision	recall	f1-score	support
0	1.00	1.00	1.00	10
1	1.00	1.00	1.00	9
2	1.00	1.00	1.00	11
accuracy			1.00	30
macro avg	1.00	1.00	1.00	30
weighted avg	1.00	1.00	1.00	30

Figure 10.2: Classification Report generated after model evaluation.

11. CONCLUSION

The Iris Flower Classification project successfully demonstrated the complete workflow of a supervised machine learning classification problem. Using the Python programming language and the Scikit-learn library, a Decision Tree Classifier was developed to classify Iris flowers into three different species based on their morphological measurements.

The project included loading the dataset, preparing the input and target variables, splitting the dataset into training and testing sets, training the machine learning model, generating predictions, and evaluating performance.

The trained model achieved **100% prediction accuracy** on the testing dataset, demonstrating that the selected algorithm was highly effective for this classification task.

This project also provided valuable practical experience in applying Python programming and machine learning concepts to solve a real-world classification problem. It strengthened the understanding of supervised learning, model training, prediction, and performance evaluation.

Overall, the project successfully met all the stated objectives and provided a strong foundation in Data Science and Machine Learning.

13. FUTURE SCOPE

Although this project successfully classified Iris flowers using a Decision Tree algorithm, future improvements can enhance the overall performance and applicability of the system.

Possible future enhancements include:

- Implementing additional machine learning algorithms such as Random Forest, Support Vector Machine (SVM), and K-Nearest Neighbors (KNN).
- Comparing the performance of multiple classification algorithms.
- Visualizing the dataset using graphs and charts.
- Deploying the trained model as a web application.
- Using larger and more complex datasets for real-world applications.
- Integrating the model with biological research and bioinformatics applications.

14. REFERENCES

1. Pedregosa, F., et al. (2011). *Scikit-learn: Machine Learning in Python*. Journal of Machine Learning Research.
2. Fisher, R. A. (1936). *The Use of Multiple Measurements in Taxonomic Problems*.
3. Python Software Foundation. *Python Programming Language*.
4. Scikit-learn Documentation.
5. CodeAlpha Data Science Internship Guidelines.